

Aula 12: Transformers, Atenção e BERT

A Revolução da Atenção e dos Modelos de Linguagem Pré-treinados

Eliezer de Souza da Silva
sereliezer.github.io
eliezer.silva@ufc.br

Mestrado e Doutorado em Ciência da Computação (MDCC)
Universidade Federal do Ceará

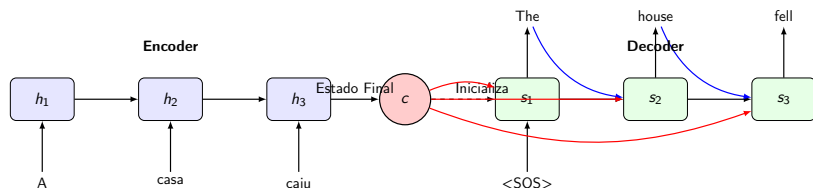
Novembro de 2025

Roteiro da Aula

- 1 Recap: O Gargalo do Seq2Seq
- 2 O Mecanismo de Atenção
- 3 O Transformer: “Attention Is All You Need”
- 4 BERT: O Início da Era dos PLMs
- 5 Além do BERT: A Evolução Continua
- 6 Encerramento

Na Aula Anterior... I

- Exploramos a arquitetura **Encoder-Decoder** para tarefas de sequência para sequência (Seq2Seq).
- O Encoder processa a sequência de entrada e a resume em um único vetor de contexto de tamanho fixo (c).
- O Decoder usa esse vetor c para gerar a sequência de saída.
- Vimos que fornecer c a cada passo do decoder melhora o desempenho.



Na Aula Anterior... II

A Limitação Fundamental

Toda a informação da sequência de entrada, não importa o seu comprimento, **deve ser comprimida** neste único vetor c . Isso cria um **gargalo de informação**.

O Problema do Vetor de Contexto Estático I

O “Thought Vector”

A ideia de um único “vetor de pensamento” (c) que encapsula o significado de uma frase inteira é poderosa, mas problemática.

- **Perda de Informação:** Para sequências longas, é quase impossível que o modelo preserve todos os detalhes importantes, especialmente do início da sequência.
- **Falta de Foco:** Ao traduzir uma frase, um humano foca em palavras específicas da fonte enquanto escreve uma palavra específica no alvo. O vetor c oferece um resumo “desfocado” da frase inteira, sem dar mais importância a nenhuma parte.

O Problema do Vetor de Contexto Estático II

“A casa azul que fica no fim da
rua, perto daquela padaria que
foi reformada ano passado,
desabou.” c

O Problema do Vetor de Contexto Estático III

Pergunta Chave

Em vez de forçar o modelo a olhar para um único resumo estático, poderíamos permitir que o decoder **prestasse atenção** a diferentes partes da entrada a cada passo da geração?

A Ideia Central: Atenção (Attention) I

Inspirado na Cognição Humana

Quando você traduz uma frase, ouve uma pergunta ou descreve uma imagem, você não processa a informação inteira de uma vez. Você foca sua atenção em partes relevantes da entrada à medida que produz a saída.

A ideia do mecanismo de atenção (Bahdanau et al., 2014) é dar essa capacidade ao modelo.

A Ideia Central: Atenção (Attention) II

Can you help this sentence to translate

Kannst du mir helfen diesen Satz zu uebersetzen ?

This diagram illustrates monotonic alignment. It consists of two rows of text. The top row is the English sentence "Can you help this sentence to translate" with the words "Can", "you", "help", "this", "sentence", "to", and "translate" highlighted in red. The bottom row is the German sentence "Kannst du mir helfen diesen Satz zu uebersetzen ?". Vertical black arrows point from each word in the German sentence to its corresponding word in the English sentence: "Kannst" to "Can", "du" to "you", "mir" to "help", "helfen" to "this", "diesen" to "sentence", "Satz" to "to", and "zu uebersetzen ?" to "translate".

Can you help me to translate this sentence

Kannst du mir helfen diesen Satz zu uebersetzen ?

This diagram illustrates non-trivial alignment. It consists of two rows of text. The top row is the English sentence "Can you help me to translate this sentence" with the words "Can", "you", "help", "me", "to", "translate", "this", and "sentence" highlighted in green. The bottom row is the German sentence "Kannst du mir helfen diesen Satz zu uebersetzen ?". Arrows show the following connections: "Kannst" to "Can", "du" to "you", "mir" to "me", and "helfen" to "help". From "diesen", three arrows branch out to "to", "translate", and "this". From "Satz", two arrows branch out to "to" and "translate". From "zu uebersetzen ?", one arrow branches out to "translate" and "sentence".

Figura: Em cima: alinhamento monótono. Em baixo: alinhamento não-trivial que a atenção consegue capturar.

A Ideia Central: Atenção (Attention) III

Como Funciona (Alto Nível)

- 1 O Encoder produz uma sequência de estados ocultos $(h_1, h_2, \dots, h_S)$, um para cada token de entrada. **Não descartamos mais os estados intermediários!**
- 2 A cada passo da decodificação (t), o decoder gera um “vetor de consulta” (s_{t-1}) que representa “o que eu preciso saber agora?”.
- 3 O mecanismo de atenção compara essa consulta com todos os estados do encoder (h_j) para calcular **pesos de atenção** (α_{tj}) .
- 4 Esses pesos são usados para criar um **vetor de contexto dinâmico** (c_t) , que é uma soma ponderada dos estados do encoder.
- 5 O decoder usa c_t para prever a próxima palavra, y_t .

Atenção: Mecânica I

Para cada passo de tempo t do decoder, com estado oculto s_{t-1} :

- 1 Calcular Scores de Alinhamento (e_{tj}):** Compara a consulta do decoder (s_{t-1}) com cada estado do encoder (h_j) para obter um score.
- 2 Calcular Pesos de Atenção (α_{tj}):** Aplica um softmax sobre os scores para obter uma distribuição de probabilidade.
- 3 Calcular o Vetor de Contexto (c_t):** Calcula a soma ponderada dos estados do encoder usando os pesos de atenção.
- 4 Gerar a Saída:** O decoder usa o contexto c_t para prever a próxima palavra.

Atenção: Mecânica II

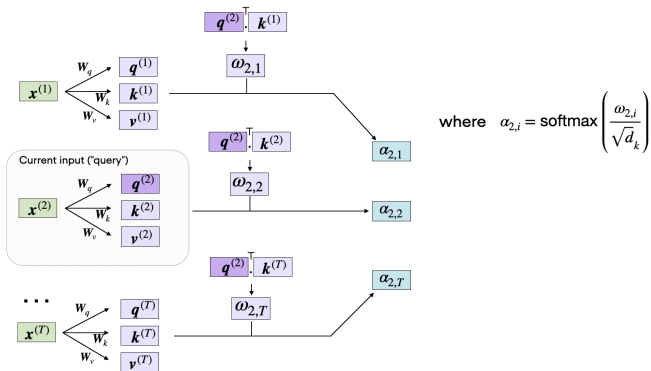


Figura: Cálculo dos scores (ω) e dos pesos de atenção (α) para a consulta atual ($x^{(2)}$) em relação a todas as entradas.

Atenção: Passos Detalhados I

Para cada passo de tempo t do decoder, com estado oculto s_{t-1} :

1. Calcular Scores de Alinhamento

Calculamos um score e_{tj} que mede o quão bem a entrada na posição j e a saída na posição t combinam. Uma função de score comum é um produto escalar.

$$\text{score}(s_{t-1}, h_j) = s_{t-1}^\top W_a h_j \quad \forall j \in \{1, \dots, S\}$$

Atenção: Passos Detalhados II

2. Calcular Pesos de Atenção (Softmax)

Normalizamos os scores usando uma função softmax para obter uma distribuição de probabilidade sobre as palavras de entrada. Estes são os pesos α_{tj} .

$$\alpha_{tj} = \frac{\exp(\text{score}(s_{t-1}, h_j))}{\sum_{k=1}^S \exp(\text{score}(s_{t-1}, h_k))}$$

Note que $\sum_{j=1}^S \alpha_{tj} = 1$.

Atenção: Passos Detalhados III

3. Calcular o Vetor de Contexto Dinâmico

O contexto c_t é a soma ponderada dos estados ocultos do encoder.

$$c_t = \sum_{j=1}^S \alpha_{tj} h_j$$

4. Gerar a Saída

O decoder usa este novo contexto c_t (geralmente concatenado com seu próprio estado s_{t-1}) para produzir a próxima palavra y_t .

$$p(y_t | y_{<t}, x) \propto f(s_{t-1}, y_{t-1}, c_t)$$

Visualizando a Atenção I

O Que a Atenção nos Oferece?

- **Alívio do Gargalo:** O decoder agora tem acesso a todos os estados do encoder em cada passo. Nenhuma informação é descartada.
- **Interpretabilidade:** Os pesos de atenção (α_{tj}) nos mostram em quais palavras de entrada o modelo está "focando" para gerar uma determinada palavra de saída. Isso pode revelar alinhamentos linguísticos interessantes.
- **Melhor Desempenho:** Especialmente em sequências longas, a atenção superou drasticamente os modelos Seq2Seq com contexto estático.

Visualizando a Atenção II

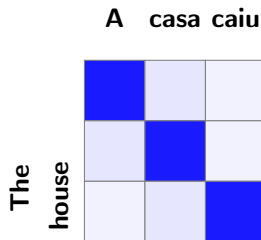


Figura: Exemplo de uma matriz de atenção para tradução. A intensidade da cor indica o peso α_{tj} .

A Próxima Revolução I

Pergunta Radical (Vaswani et al., 2017)

Vimos que a atenção melhora muito os modelos Seq2Seq baseados em RNNs. Mas... e se nos livrarmos completamente das RNNs e construirmos um modelo **apenas com mecanismos de atenção?**

A Próxima Revolução II


 Cornell University

We gratefully acknowledge support from the Simons Foundation and member institutions.

arXiv > cs > arXiv:1706.03762

Search... All Fields Search

Help | Advanced Search

Computer Science > Computation and Language

[Submitted on 12 Jun 2017 (v1), last revised 6 Dec 2017 (this version, v5)]

Attention Is All You Need

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks in an encoder-decoder configuration. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

Download:

- PDF
- Other formats

Download

Current browse context: cs.CL

< prev | next >

new | recent | 1706

Change to browse by:

cs

cs.LG

References & Citations

- NASA ADS
- Google Scholar
- Semantic Scholar

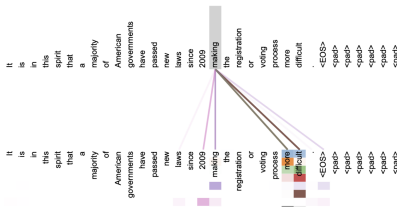
84 blog links [what's new](#)

DBLP - CS Bibliography

listing | bibtext

Ashish Vaswani
Noam Shazeer
Niki Parmar
Jakob Uszkoreit

Attention Visualizations



A Inovação Central: Self-Attention I

Olhando para a Própria Sentença

Até agora, a atenção era entre o decoder e o encoder. A **Self-Attention** permite que os inputs interajam entre si. Cada palavra na sentença de entrada pode “prestar atenção” a todas as outras palavras na **mesma sentença**.

A Inovação Central: Self-Attention II

Por que isso é útil?

Considere a frase: "O animal não atravessou a rua porque **ele** estava muito cansado."

- Para entender a quem "ele" se refere, o modelo precisa conectá-lo a "animal".
- A Self-Attention calcula um peso de atenção entre "ele" e todas as outras palavras, e (idealmente) aprenderá a dar um peso alto para "animal".
- Isso enriquece a representação de cada palavra com informações contextuais de toda a sentença.

Queries, Keys e Values (Q, K, V) I

A Mecânica da Auto-Atenção

Para permitir que as palavras interajam, cada vetor de embedding de entrada (x_i) é projetado em três novos vetores através da multiplicação por matrizes de pesos (W^Q , W^K , W^V) que são aprendidas durante o treino.

- **Query** ($Q = \{q_i\}$): Representa a palavra atual, a sondar as outras.
- **Key** ($K = \{k_i\}$): Um "rótulo" para a palavra, que pode ser correspondido por uma query.
- **Value** ($V = \{v_i\}$): A informação real da palavra, a ser passada adiante se houver correspondência.

Queries, Keys e Values (Q, K, V) II

$$\begin{matrix} \text{X} \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} \end{matrix} \times \begin{matrix} W^Q \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} \end{matrix} = \begin{matrix} Q \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array} \end{matrix}$$

$$\begin{matrix} \text{X} \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} \end{matrix} \times \begin{matrix} W^K \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} \end{matrix} = \begin{matrix} K \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array} \end{matrix}$$

$$\begin{matrix} \text{X} \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} \end{matrix} \times \begin{matrix} W^V \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline & & & \\ \hline \end{array} \end{matrix} = \begin{matrix} V \\ \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array} \end{matrix}$$

Queries, Keys e Values (Q, K, V) III

Figura: Geração das matrizes Query, Key e Value a partir da matriz de entrada X .

A Matemática: Scaled Dot-Product Attention I

A pontuação de atenção entre a palavra i e a palavra j é o produto escalar de suas queries e keys.

A Matemática: Scaled Dot-Product Attention II

A Fórmula

Para uma matriz de entradas X , criamos as matrizes Q, K, V multiplicando X por matrizes de pesos aprendidas (W_Q, W_K, W_V).

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

A saída da camada de atenção é então calculada como:

$$\text{Attention}(Q, K, V) = \underbrace{\text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)}_{\text{Pesos de Atenção}} V$$

A Matemática: Scaled Dot-Product Attention III

- O termo $\sqrt{d_k}$ é um fator de escala (onde d_k é a dimensão dos vetores de key). Ele é crucial para a estabilidade do gradiente durante o treinamento.
- O resultado é uma nova representação para cada palavra, que é uma soma ponderada dos vetores de *Value* de toda a sentença, onde os pesos são determinados pela compatibilidade entre *Query* e *Key*.

A Matemática: Scaled Dot-Product Attention IV

$$\begin{aligned}
 & \text{softmax} \left(\frac{\begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{K}^T \\ \begin{array}{|c|c|} \hline \square & \square \\ \hline \square & \square \\ \hline \square & \square \\ \hline \end{array} \end{matrix} \right) \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \\
 & = \begin{matrix} \text{Z} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}
 \end{aligned}$$

Figura: Cálculo da matriz de saída Z da camada de self-attention.

Multi-Head Attention I

Múltiplos Pontos de Vista

Em vez de calcular a atenção uma vez, o Transformer faz isso várias vezes em paralelo com diferentes matrizes de pesos W^Q , W^K , W^V . Cada uma dessas "cabeças de atenção" pode aprender a focar em diferentes tipos de relações (sintáticas, semânticas, etc.). Os resultados são depois concatenados e projetados novamente.

Multi-Head Attention II

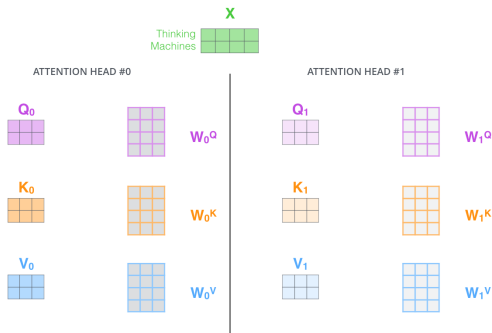
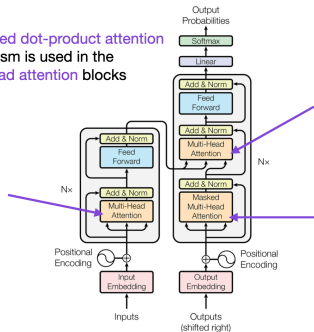


Figura: Cada “cabeça de atenção” tem as suas próprias matrizes de pesos W , gerando diferentes conjuntos de Q , K e V .

Arquitetura do Transformer I

The **scaled dot-product attention** mechanism is used in the **multi-head attention blocks**



Source: "Attention Is All You Need" (<https://arxiv.org/abs/1706.03762>)

Figura: A arquitetura completa do Transformer, do paper original.

Arquitetura do Transformer II

Elementos I

- **Multi-Head Attention:** Em vez de calcular a atenção uma vez, o Transformer faz isso várias vezes em paralelo (com diferentes matrizes W_Q , W_K , W_V). Cada “cabeça de atenção” pode aprender a focar em diferentes tipos de relações (sintáticas, semânticas, etc.). Os resultados são concatenados.
- **Positional Encodings:** Como não há RNNs, o modelo não tem noção da ordem das palavras. Injetamos essa informação adicionando um "vetor de posição" aos embeddings de entrada. O artigo original usou funções de seno e cosseno.

Arquitetura do Transformer III

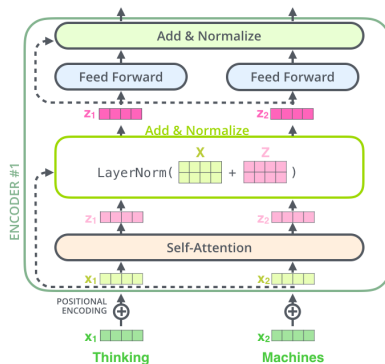


Figura: Fluxo de dados através de uma sub-camada, mostrando a conexão residual (soma) e a normalização de camada.

Arquitetura do Transformer IV

Elementos II

- **Feed-Forward Networks:** Após a atenção, cada posição passa por uma rede feed-forward simples, mas independente.
- **Residual Connections & Layer Norm:** Técnicas padrão para ajudar a treinar redes muito profundas.
- **Encoder-Decoder Stacks:** O Transformer completo tem uma pilha de Encoders e uma pilha de Decoders, seguindo o paradigma Seq2Seq, mas usando esses novos blocos.

Arquitetura do Transformer V

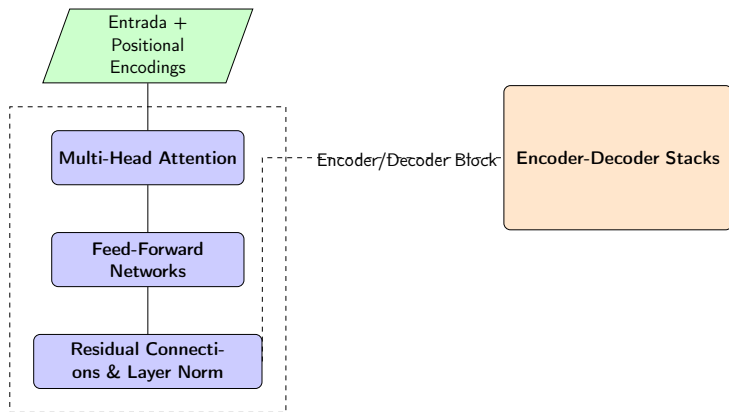


Figura: Uma representação simplificada dos principais componentes do Transformer.

Uma Mudança de Paradigma: Pré-treinamento e Fine-Tuning I

O Problema da Escassez de Dados

Treinar modelos complexos como o Transformer do zero para uma tarefa específica (ex: análise de sentimentos) requer uma quantidade enorme de dados rotulados.

Uma Mudança de Paradigma: Pré-treinamento e Fine-Tuning II

A Solução em Duas Etapas

- 1 Pré-treinamento (Pre-training):** Use um corpus massivo de texto **não rotulado** para treinar um modelo de linguagem de propósito geral. O modelo aprende a "entender" a linguagem.
- 2 Ajuste Fino (Fine-tuning):** Pegue o modelo pré-treinado e adapte-o para uma tarefa específica de "downstream" usando os seus dados rotulados (que podem ser poucos).

BERT: Bidirectional Encoder Representations from Transformers I

O que é o BERT? (Devlin et al., 2018)

- BERT é, essencialmente, a **pilha de Encoders** da arquitetura Transformer.
- Sua principal inovação foi ser o primeiro modelo de linguagem **profundamente bidirecional**.
- Graças à Self-Attention, cada palavra em BERT pode ver todas as outras palavras na sentença, desde a primeira camada, permitindo um entendimento contextual muito mais rico.

BERT: Bidirectional Encoder Representations from Transformers II

Como Pré-treinar um Modelo Bidirecional?

Não podemos simplesmente prever a próxima palavra, porque o modelo pode "trapacear" e olhar para a resposta. BERT introduz duas tarefas de pré-treinamento engenhosas.

Objetivos de Pré-treinamento do BERT I

1. Masked Language Model (MLM)

- Inspirado no "Teste Cloze".
- Antes de passar a sentença para o modelo, mascaramos aleatoriamente 15% dos tokens.
- **Tarefa:** O modelo deve prever o token original que foi mascarado, com base no contexto não mascarado de ambos os lados.
- Exemplo: '[CLS] O [MASK] pulou sobre o cachorro. [SEP]'
- O modelo é forçado a fundir o contexto esquerdo e direito para fazer uma previsão.

Objetivos de Pré-treinamento do BERT II

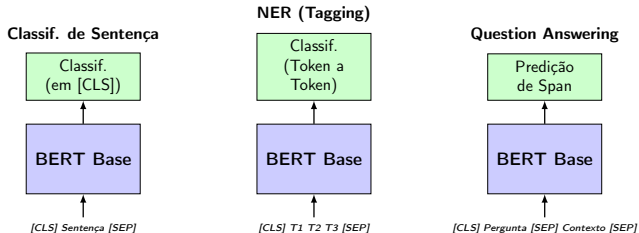
2. Next Sentence Prediction (NSP)

- O modelo recebe dois segmentos de texto, A e B, e deve prever se B é a sentença que realmente se segue a A no corpus original.
- Exemplo:
 - (A: O homem foi à loja. B: Ele comprou um litro de leite.) → **IsNext**
 - (A: O homem foi à loja. B: Pinguins não voam.) → **NotNext**

Como Usar o BERT: Fine-Tuning I

Adaptando para Tarefas Específicas

Após o pré-treinamento, o BERT pode ser adaptado para várias tarefas de NLP com modificações mínimas, adicionando uma pequena camada no topo.



RoBERTa: A Robustly Optimized BERT Pretraining Approach I

O BERT foi treinado da melhor forma?

Pesquisadores do Facebook AI (Liu et al., 2019) revisitaram o BERT e descobriram que seu desempenho poderia ser significativamente melhorado.

RoBERTa: A Robustly Optimized BERT Pretraining Approach II

Principais Mudanças do RoBERTa

- **Remoção do NSP:** Descobriram que a tarefa de Next Sentence Prediction era mais prejudicial do que benéfica e a removeram.
- **Masking Dinâmico:** Em vez de uma máscara estática, uma nova máscara é gerada a cada vez que uma sequência é alimentada ao modelo.
- **Mais Dados, Mais Tempo:** RoBERTa foi treinado com uma quantidade de dados muito maior (160GB vs 16GB do BERT) e por mais tempo.

RoBERTa: A Robustly Optimized BERT Pretraining Approach III

Resultado

RoBERTa estabeleceu um novo estado da arte, mostrando que o BERT original estava "sub-treinado" e que a receita de treinamento é tão importante quanto a arquitetura.

Resumo e Próximos Passos I

O que Vimos Hoje

- O problema do **gargalo de informação** levou ao desenvolvimento do **mecanismo de atenção**.
- A arquitetura **Transformer** eliminou a recorrência, usando **Self-Attention**.
- O **paradigma de pré-treinamento e fine-tuning** revolucionou a PNL.
- **BERT** usou o encoder do Transformer e objetivos de pré-treinamento (MLM, NSP) para alcançar um entendimento bidirecional da linguagem.
- Modelos como **RoBERTa** refinaram a receita de treinamento, alcançando resultados ainda melhores.

Resumo e Próximos Passos II

Na Próxima Aula

Agora que temos esses modelos poderosos, quais são os desafios? Discutiremos eficiência, considerações éticas, e a ascensão de modelos ainda maiores e mais capazes, como a família GPT.

Referências I